theory exe1

1. Introduction to Python and its Features

Python is a programming language, but its biggest advantage is how incredibly easy it is to learn and use.

- **Simple:** The syntax (the way we write the code) looks very much like normal English. You can usually read it and guess what the code is trying to do.
- **High-Level:** This means we don't have to worry about the complex language that the computer hardware understands. We write code in a way that is easy for humans, and Python translates it for the computer.
- **Interpreted:** Python reads and runs the code line-by-line. Because of this, if there is a mistake (an error) in your code, Python tells you exactly which line has the problem, making it very easy to fix.

2. History and Evolution of Python

Python was created by a man named Guido van Rossum in 1991. He was looking for a hobby project during his Christmas holidays and wanted to create a language that was both easy to read and fun to use.

- *Fun Fact:* If you are wondering if Python is named after the big snake, it actually isn't! Guido named it after a famous old British comedy show called "Monty Python's Flying Circus" because he was a big fan.

3. Advantages of Using Python

Why is Python so popular compared to other languages?

- **Easy to Learn:** As mentioned, it is very beginner-friendly. The code is much shorter and cleaner.
- **Saves Time:** A program that might take 10 to 15 lines of code in languages like C++ or Java can often be written in just 2 or 3 lines in Python.
- **Huge Community:** Because so many people use Python, it is very easy to find help online. If you ever get stuck, someone on the internet has probably already solved that exact problem.
- **Built-in Libraries:** You don't have to build everything from scratch. Python has pre-made tools (called libraries) for almost everything, whether you want to build a website, create AI, or work in cybersecurity.

4. Installing Python and Setting Up the Environment

To start coding, you need to install Python on your computer and download a software (called an IDE) to write your code in.

- **Installing Python:** You can download it for free from python.org. *(Important Note: When installing, make sure to check the small box that says "Add Python to PATH" at the bottom of the screen!)*

- **Where to write code (IDE Options):**
    - **VS Code (Visual Studio Code):** This is the most popular choice. It is lightweight, fast, and you can add extensions to customize it however you want.
    - **PyCharm:** This software is slightly heavier, but it is made specifically for Python. It has a lot of helpful features built right into it.
    - **Anaconda:** If you plan on studying Data Science or AI in the future, Anaconda is great because it comes with all the heavy tools and libraries pre-downloaded.

5. To write my first py progeramme:

```
print(“hello world”)
```

theory exe 2

Understanding Python's PEP 8 guidelines is essential for any programmer. PEP stands for Python Enhancement Proposal, and PEP 8 serves as the official style guide or rulebook for writing Python code. When many different people write code, everyone naturally has their own style, which can make programs very difficult to read. PEP 8 solves this problem by providing a standard set of rules so that all Python code looks clean, similar, and consistent, regardless of who actually wrote it.

Following PEP 8 involves specific rules for indentation, comments, and naming conventions. In Python, empty space or indentation is used to organize blocks of code instead of using brackets. The standard rule is to always use exactly four spaces for each indentation level and to never mix tabs with spaces. Comments are also a crucial part of coding. They use the hash symbol and should be used to explain why a piece of code is written, rather than just stating what it does. Furthermore, PEP 8 provides clear naming conventions so programmers can instantly recognize different parts of the code. For example, variables and functions should be written in all lowercase letters with words separated by underscores, which is known as snake case. Classes should start with a capital letter without any underscores, known as camel case, while constant values should always be written in all capital letters.

Ultimately, applying all these rules helps developers write readable and maintainable code. There is a common rule in programming that code is read much more often than it is written. Writing maintainable code means that if you or someone else looks at your project months later, it is still easy to understand, update, and fix without frustration. To achieve this, programmers must use meaningful and descriptive names for their variables instead of single letters. It is also highly recommended to keep lines of code relatively short, use blank lines to separate different sections like paragraphs in an essay, and always focus on simple, step-by-step logic rather than trying to write unnecessarily complex code.

theri exe 3

In Python, data types determine the specific kind of information being stored. These include fundamental types like integers for whole numbers, floats for decimals, and strings for text. Python also offers versatile collections to manage multiple items: lists for changeable data, tuples for unchangeable data, dictionaries for key-value pairs, and sets for storing unique items. Variables serve as simple names or labels attached to this data. Python handles memory allocation dynamically, automatically assigning the right amount of space when a variable is created, while a built-in garbage collection system efficiently clears out unused memory to keep the program running smoothly. To manipulate this data, Python utilizes several operators. Arithmetic operators perform basic math calculations, comparison operators evaluate values against each other, logical operators combine conditions for complex decisions, and bitwise operators handle advanced, low-level binary calculations.

theoruy exe 4

onditional statements in Python allow a program to make decisions based on specific rules, much like how humans make choices in real life. The core of this decision-making process involves 'if', 'elif', and 'else' statements. An 'if' statement checks if a certain condition is true; if it is, the program runs the code inside that block. If the initial condition is false, the program can move to an 'elif' (which stands for "else if") statement to check a new, alternative condition. Finally, the 'else' statement acts as a catch-all that runs its code only when all previous 'if' and 'elif' conditions turn out to be false.

Sometimes, decisions require multiple steps, which is where nested if-else conditions are used. Nesting simply means placing one conditional statement inside another. This allows the program to check a secondary condition only after a primary condition has already been evaluated as true. By using nested conditions, programmers can build step-by-step logical paths and handle much more complex and detailed decision-making scenarios within their code.

theory exe 5

ntroduction to Loops in Python

In computer science, a **loop** is a fundamental control flow structure designed to execute a block of instructions repeatedly as long as a specified condition is met. The primary purpose of loops is to automate repetitive tasks, reduce code redundancy, and handle data streams efficiently without manual duplication of statements.

Python relies on two primary types of loop structures:

- **The for Loop:** This structure handles **definite iteration**, meaning the number of repetitions is predetermined before execution. It is designed to step through a sequence or an iterable object sequentially, executing the block of code exactly once for each item.
- **The while Loop:** This structure handles **indefinite iteration**, meaning the total number of repetitions is not known beforehand. It relies entirely on a conditional expression, continuing execution continuously as long as that condition evaluates to true.

Theoretical Framework: How Loops Work in Python

The underlying mechanics of loops in Python differ significantly from lower-level languages like C or Java, primarily due to Python's approach to memory and object tracking.

1. Structure and Indentation

Unlike languages that use curly braces {} to define the scope of a loop, Python utilizes strict horizontal spacing (indentation). A colon : marks the end of the loop header, and every subsequent line indented at the same level is considered part of the loop body. The loop terminates as soon as the interpreter encounters a line matching the indentation level of the loop header.

2. The Mechanics of a for Loop

Under the hood, a Python for loop does not simply increment a counter variable. Instead, it utilizes the **Iterator Protocol**.

- When a for loop initializes, it requests an internal tracker called an **iterator** from the target sequence.
- The loop automatically calls a built-in fetching mechanism to retrieve the next consecutive element.
- This process continues seamlessly until the sequence runs out of items and raises a built-in termination signal, telling the interpreter to gracefully exit the loop.

3. The Mechanics of a while Loop

A while loop operates purely on **Boolean evaluation**.

- Before entering the loop body, the interpreter checks the designated conditional expression.
- If the expression evaluates to true, the control flow moves inside the block.
- Once the block completes, the program jumps back to the header to re-evaluate the condition.
- If the condition ever evaluates to false, the loop immediately stops, and execution resumes on the next line outside the block.

Theoretical Integration with Collections

Python's design makes loops heavily integrated with data structures like lists, tuples, sets, and dictionaries. In theoretical terms, these collections are classified as **iterables**—objects capable of returning their elements one at a time.

1. Element-by-Element Traversal

Instead of accessing memory addresses or indexing positions manually, Python loops use direct assignment. During traversal, a placeholder variable defined in the loop header acts as a dynamic reference. With each rotation of the loop, this variable automatically re-binds itself to reference the next memory object contained inside the collection.

2. Mutability vs. Immutability in Traversal

- **Lists (Mutable Collections):** When looping through a list, the values can theoretically be modified dynamically during execution, though altering the structure of a list while iterating over it can cause synchronization issues in memory tracking.
- **Tuples (Immutable Collections):** Traversal occurs in the exact same logical order as a list, but because a tuple cannot be changed after creation, the data read during the loop remains secure and protected against accidental modifications.

theory exe 7

Here is the next section of your assignment text. It covers function definitions, arguments, variable scope, and the theoretical classification of built-in methods, kept completely free of code snippets as requested.

Defining and Calling Functions in Python

In computer programming, a **function** is a self-contained, reusable block of structured code designed to perform a single, cohesive task. Functions are the foundational building blocks of modular programming, allowing developers to break complex problems into smaller, manageable, and traceable sub-tasks.

1. Function Definition

Defining a function involves establishing its blueprint in memory. The process begins with a specific declaration keyword (def), followed by a unique identifier (the function name) and a pair of parentheses which can optionally house parameters. A colon terminates the header line, indicating that the subsequent indented block forms the local environment and operational logic of that function. The definition phase merely registers the function; no execution occurs at this stage.

2. Function Invocation (Calling)

Calling or invoking a function transfers the program’s control flow from the main execution line directly into the function’s internal block. To trigger invocation, the program uses the function's name followed by parentheses containing the necessary arguments. Once the function executes its statements, control flow is returned back to the exact point in the program where the call was made. Functions can communicate data back to the calling environment using a termination statement that passes an evaluation value or object.

Theoretical Architecture of Function Arguments

Arguments act as the data bridge between the outside program and the local environment of a function. Python categorizes arguments based on how they are bound to the function's internal parameters during invocation.

1. Positional Arguments

Positional arguments are the most fundamental form of data passing. They are bound strictly by their order and relative placement during the function call. The first argument passed maps directly to the first parameter defined, the second to the second, and so forth. The total count and structural sequence of arguments must match the definition exactly, or the interpreter throws a structural mismatch error.

2. Keyword Arguments

Keyword arguments remove dependency on structural ordering. During invocation, each argument is explicitly linked to its corresponding parameter name using an explicit assignment indicator. Because the parameter identity is explicitly stated, the arguments can be passed in any random order without corrupting the function's internal logic.

3. Default Arguments

Default arguments allow parameters to possess a predetermined fallback value within the function's definition header. If the calling environment fails to provide an argument for that specific parameter during invocation, the function automatically initializes it with the default value. This mechanism allows functions to remain highly flexible, supporting both simplified standard calls and highly customized overrides.

Scope of Variables in Python

Scope refers to the specific region of a program where a variable is recognized, accessible, and valid for memory operations. Python resolves variable visibility and name identification using a strict hierarchical framework known as the **LEGB Rule**.

- **Local Scope:** Variables declared inside the body of a function exist solely within that function's execution lifecycle. They are created upon function invocation and entirely destroyed from memory once the function terminates, rendering them completely inaccessible to the outside program.
- **Enclosing Scope:** This scope exists only within nested functions. If a function is defined inside another function, the inner function has read-only access to the variables belonging to the outer, enveloping function's environment.
- **Global Scope:** Global variables are declared at the topmost level of the script or module. They are initialized when the program starts and remain alive until execution completes, making them visible and readable by any function within that specific script.
- **Built-in Scope:** This is the highest tier of the hierarchy. It contains pre-loaded, foundational names and exception structures that are permanently loaded into memory by the Python interpreter and are globally available at all times without initialization.

Theoretical Overview of Built-in Methods

In object-oriented design, a **method** is a specialized function that is intrinsically bound to a specific data type or class object. Python provides robust, pre-compiled built-in methods to manipulate primary data structures efficiently.

1. String Methods

Strings in Python are fundamentally **immutable**, meaning their sequence of characters cannot be altered in place within memory. Consequently, all built-in string methods function as **transformative extractors**. When called upon a string, they perform analytical checks,

case adjustments, formatting adjustments, or substrings extractions, and seamlessly generate an entirely new string object in memory, leaving the source data pristine.

2. List Methods

Unlike strings, lists are **mutable** data structures. Built-in list methods operate via **in-place manipulation**. They directly alter the internal structural layout and memory references of the existing list object. These methods handle structural tasks such as inserting new elements at precise index positions, appending items to the tail, removing target values, or reordering the underlying elements chronologically or alphabetically without generating a secondary collection.

theory exe 8

The Role of Loop Control Statements in Python

In computer programming, loop control statements alter the standard, sequential execution flow of a loop. Under normal circumstances, a loop runs continuously through its entire collection or until its conditional expression evaluates to false.

Python provides three distinct keywords—break, continue, and pass—to manually intercept, skip, or placeholder this execution cycle based on internal logical criteria.

1. The break Statement (Loop Termination)

The break statement handles **immediate termination**. When the interpreter encounters this keyword inside a loop body, it abruptly halts the iteration process, bypasses any remaining code within the current cycle, and forces the program's control flow to exit the loop entirely.

- **Mechanics:** Execution immediately jumps to the first line of code outside and directly below the loop structure.
- **Theoretical Application:** It is primarily utilized in search algorithms or input validation systems. Once a target item or condition is successfully found or met, running further iterations becomes computationally redundant. The break statement prevents unnecessary processor cycles by ending the loop early.

2. The continue Statement (Iteration Skipping)

The continue statement handles **cycle interruption**. Unlike break, it does not terminate the loop entirely. Instead, it immediately stops the execution of the *current* iteration cycle and forces the control flow back to the top of the loop header.

- **Mechanics:** Any statements placed below the continue keyword within the loop block are completely ignored for that single round. In a for loop, the iterator immediately advances to the next chronological item; in a while loop, the program goes straight back to evaluating the core conditional statement.
- **Theoretical Application:** This is used to filter out noise or specific edge cases. If a particular data element does not meet the required processing criteria, continue skips the rest of the operational logic for that specific element and shifts focus to the next available data point.

3. The pass Statement (Null Operation Placeholder)

The pass statement handles **syntactic preservation**. It is a null operation, meaning absolutely nothing happens when it executes.

- **Mechanics:** Python requires a valid block of code following any control flow header (like a loop or a conditional statement) due to its indentation-based syntax architecture. A blank space or a mere comment will result in a structural compilation error. The pass keyword satisfies this requirement by acting as a syntactical filler that

keeps the program structurally sound while performing zero memory or logic operations.

- **Theoretical Application:** It is predominantly used as a placeholder during the structural drafting of a program. It allows developers to architect empty loops, abstract classes, or unwritten function blocks to establish the overall program framework before diving into writing the actual implementation details.

theory exe 9

Here is the theoretical breakdown of string architecture, operations, and slicing in Python, structured for your assignment.

Theoretical Architecture of Strings in Python

In computer science, a **string** is a contiguous sequence of characters treated as a single data object. In Python, strings are classified as immutable sequences. **Immutability** means that once a string object is allocated a specific block of memory, the characters within that sequence cannot be altered, added, or reordered in place. Any operation that appears to modify a string actually constructs an entirely new string object elsewhere in memory.

Foundational String Operations

Python provides a set of built-in operators and structural methods to interact with and transform text data.

1. Concatenation and Repetition

- **Concatenation:** This operation involves joining two or more independent string sequences end-to-end to form a single, unified string. It uses a binary addition operator to instruct the interpreter to allocate a new memory space large enough to hold the combined character lengths of the source strings sequentially.
- **Repetition:** This operation uses a multiplication operator between a string and an integer. It instructs the memory manager to duplicate the sequence a specific number of times consecutively, returning the expanded sequence as a new string object.

2. Method-Driven Structural Transformations

Because strings are objects, they carry pre-compiled methods that perform algorithmic checks and transformations. Since strings are immutable, these methods do not change the source data; they return fresh, modified copies:

- **Case Modification Methods:** Operations like case-raising or case-lowering read the ASCII or Unicode values of individual characters. If a character falls within the lowercase range, the system shifts its binary value to output the uppercase equivalent, and vice versa, while completely ignoring non-alphabetic characters.
- **Stripping and Padding Methods:** These methods scan the boundaries of a sequence (the leading and trailing edges) to detect and purge whitespace characters or specified padding indicators, optimizing strings for clean data storage and matching.

The Mechanics of String Slicing

Slicing is a sophisticated mechanism that extracts a specific subsection (a substring) from a parent sequence without modifying the original data structure.

1. The Indexing Grid

Python maps every character in a string to a distinct numerical coordinate based on its position, starting from zero. Python supports two parallel indexing tracks:

- **Positive Indexing:** Travels from left to right, beginning at index 0 for the first character and ending at $N-1$ (where N is the total length of the string).
- **Negative Indexing:** Travels from right to left, beginning at index -1 for the terminal character and counting backwards. This allows programs to reference the end of a string dynamically without calculating its absolute length.

2. The Three-Parameter Slicing Framework

Slicing operates on an algebraic notation system that accepts up to three step-control parameters separated by colons: [Start : Stop : Step].

- **The Start Bound:** Defines the exact index position where the substring extraction sequence begins. If left blank, it defaults to the absolute beginning of the string.
- **The Stop Bound (Exclusive Boundary):** Specifies the index where the extraction must halt. Crucially, this boundary is **exclusive**, meaning the character sitting exactly at the stop index is excluded from the final substring. The slice stops exactly one position prior ($Stop - 1$). If omitted, the slice extracts all the way to the end of the sequence.
- **The Step Value (Stride):** Dictates the structural interval and direction of the traversal. A step of 1 reads every consecutive character. A step of 2 skips every alternate character.
    - *Directional Control:* A positive step moves the selection window forward (left to right). A negative step reverses the traversal mechanics, causing the interpreter to read the string backward (right to left), which is a common theoretical mechanism used for sequence reversal.